

11. ПАРАЛЛЕЛИЗМ И КОНКУРЕНТНОСТЬ (CONCURRENCY AND PARALLELISM)

Этой главой я надеюсь дать вам всю информацию и инструменты, которые понадобятся, чтобы эффективно пользоваться конкурентностью (concurrency) в ваших программах на Rust, реализовывать поддержку конкурентного использования в ваших библиотеках и правильно применять примитивы конкурентности Rust. Я не буду напрямую учить вас тому, как реализовать конкурентную структуру данных или написать высокопроизводительное конкурентное приложение. Вместо этого моя цель — дать вам достаточное понимание базовых механизмов, чтобы вы могли сами управлять ими для чего угодно, что вам понадобится.

Конкурентность бывает трёх видов: однопоточная конкурентность (как с `async/await`, что мы обсуждали в главе 10), одноядерная многопоточная конкурентность и многоядерная конкурентность, которая даёт настоящий параллелизм (parallelism).

Каждый вид позволяет по-разному чередовать выполнение конкурентных задач в вашей программе. Существует ещё больше подвидов, если учитывать детали планирования и вытеснения в операционной системе, но мы не будем углубляться в это слишком сильно.

На уровне типов Rust представляет лишь один аспект конкурентности: многопоточность. Тип либо безопасен для использования более чем одним потоком, либо нет. Даже если ваша программа имеет несколько потоков (и потому конкурентна), но лишь одно ядро (и потому не параллельна), Rust должен предполагать, что при наличии нескольких потоков может возникнуть параллелизм. Большинство типов и приёмов, о которых мы будем говорить, применимы одинаково независимо от того, выполняются ли два потока действительно параллельно или нет, поэтому, чтобы не усложнять язык изложения, я буду использовать слово *конкурентность* в неформальном смысле «вещи, выполняющиеся более или менее в одно и то же время» на протяжении всей этой главы. Когда различие будет важно, я буду на это

указывать.

Что особенно изящно в подходе Rust к типобезопасной многопоточности, так это то, что она является не свойством компилятора, а свойством библиотеки, которое разработчики могут расширять для построения сложных контрактов конкурентности. Поскольку потокобезопасность выражается в системе типов через реализации и границы `Send` и `Sync`, которые распространяются вплоть до прикладного кода, потокобезопасность всей программы проверяется исключительно средствами проверки типов.

Книга *The Rust Programming Language* уже охватывает большую часть основ, касающихся конкурентности, включая трейты `Send` и `Sync`, типы `Arc` и `Mutex`, а также каналы. Поэтому здесь я не буду многое из этого повторять, за исключением тех мест, где это стоит повторить конкретно в контексте какой-то другой темы. Вместо этого мы сначала рассмотрим то, что делает конкурентность сложной, и некоторые распространённые паттерны конкурентности, призванные справляться с этими трудностями. После этого мы исследуем, как взаимодействуют конкурентность и асинхронность (и как не взаимодействуют), прежде чем погрузиться в то, как использовать атомарные операции (atomic operations) для реализации конкурентных операций более низкого уровня. Наконец, я завершу главу некоторыми советами о том, как сохранить здравый рассудок при работе с конкурентным кодом.

Проблема с конкурентностью (The Trouble with Concurrency)

Прежде чем мы погрузимся в хорошие паттерны конкурентного программирования и детали механизмов конкурентности Rust, стоит потратить немного времени на понимание того, почему конкурентность вообще является сложной задачей. То есть почему нам нужны специальные паттерны и механизмы для конкурентного кода?

Корректность (Correctness)

Главная трудность в конкурентности — это координация доступа, в частности доступа на запись, к ресурсу, который разделяется между несколькими потоками. Если множество потоков хотят разделять ресурс исключительно с

целью его чтения, то это обычно легко: вы помещаете его в Arc или получаете на него ссылку `&'static`, и на этом всё готово. Но как только какой-либо поток захочет записать, возникает множество проблем, обычно в форме *гонок данных* (*data races*). Гонка данных возникает, когда один поток обновляет разделяемое состояние, в то время как второй поток также обращается к этому состоянию — либо чтобы прочесть его, либо чтобы обновить. Без дополнительных мер предосторожности второй поток может прочесть частично перезаписанное состояние, затереть части того, что записал первый поток, или вообще не увидеть запись первого потока! В общем случае все гонки данных считаются неопределённым поведением (*undefined behavior*).

Гонки данных являются частью более широкого класса проблем, который в основном — хотя и не исключительно — возникает в конкурентной среде: *состояний гонки* (*race conditions*). Состояние гонки возникает всякий раз, когда из последовательности инструкций возможны несколько исходов, зависящих от относительного времени других событий в системе. Этими событиями могут быть потоки, выполняющие определённый фрагмент кода, срабатывание таймера, приход сетевого пакета или любое другое событие, переменное во времени. Состояния гонки, в отличие от гонок данных, не являются по своей сути плохими и не считаются неопределённым поведением. Однако они являются благодатной почвой для багов, когда возникают особенно причудливые гонки, как вы увидите на протяжении этой главы.

Производительность (Performance)

Часто разработчики вводят конкурентность в свои программы в надежде повысить производительность. Или, говоря точнее, они надеются, что конкурентность позволит им выполнять больше операций в секунду в совокупности за счёт использования большего количества аппаратных ресурсов. Это может быть достигнуто на одном ядре путём выполнения одного потока, пока другой ожидает, либо на нескольких ядрах путём одновременного выполнения работы потоками, по одному на каждом ядре, что иначе происходило бы последовательно на одном ядре. Большинство разработчиков, говоря о конкурентности, имеют в виду именно второй вид прироста производительности, который часто формулируется в терминах масштабируемости (scalability). Масштабируемость в этом контексте означает «производительность этой программы масштабируется с количеством ядер», подразумевая, что если вы дадите вашей программе больше ядер, её производительность улучшится.

Хотя достижение такого ускорения возможно, это сложнее, чем кажется. Конечная цель в масштабируемости — линейная масштабируемость, при которой удвоение количества ядер удваивает объём работы, который ваша программа выполняет за единицу времени. Линейную масштабируемость также часто называют идеальной масштабируемостью. Однако в реальности немногие конкурентные программы достигают такого ускорения. Более распространено сублинейное масштабирование, при котором пропускная способность растёт почти линейно при переходе от одного ядра к двум, но добавление большего количества ядер даёт убывающую отдачу. Некоторые программы даже испытывают отрицательное масштабирование, при котором предоставление программе доступа к большему количеству ядер *уменьшает* пропускную способность, обычно из-за того, что множество потоков соперничают за некий разделяемый ресурс.

Может помочь представить группу людей, пытающихся полопать все пузырьки на листе пузырчатой плёнки — добавление большего количества людей помогает поначалу, но в какой-то момент вы получаете убывающую отдачу, поскольку теснота усложняет работу каждому отдельному человеку. Если участвующие люди особенно неэффективны, ваша группа может в итоге простоять без дела вокруг, обсуждая, как лучше подойти к задаче, и не полопать ни одного пузырька вообще! Этот вид помех между задачами, которые должны выполняться параллельно, называется *состязанием*

(*contention*) и является злейшим врагом хорошего масштабирования. Состязание может возникать разными способами, но главные нарушители — это взаимное исключение, исчерпание разделяемого ресурса и ложное разделение.

Взаимное исключение (Mutual Exclusion)

Когда только одной конкурентной задаче разрешено выполнять определённый фрагмент кода в любой момент времени, мы говорим, что выполнение этого сегмента кода взаимно исключается — если один поток выполняет его, никакой другой поток не может делать этого одновременно. Архетипический пример этого — блокировка взаимного исключения, или *мьютекс (mutex)*, которая явно обеспечивает, чтобы только один поток входил в определённую критическую секцию кода вашей программы в любой момент времени. Однако взаимное исключение может происходить и неявно. Например, если вы запустите поток для управления разделяемым ресурсом и будете отправлять ему задания по каналу `mpsc`, этот поток фактически реализует взаимное исключение, поскольку только одно такое задание выполняется за раз.

Взаимное исключение может также возникать при вызове функций операционной системы или библиотек, которые внутренне обеспечивают однопоточный доступ к критической секции. Например, на протяжении многих лет стандартный распределитель памяти требовал взаимного исключения для некоторых выделений, что делало выделение памяти операцией, влекущей значительное состязание в остальных высокопараллельных программах. Аналогично, многие операции операционной системы, которые, казалось бы, должны быть независимыми, например создание двух файлов с разными именами в одном каталоге, могут в итоге происходить последовательно внутри ядра.

ПРИМЕЧАНИЕ Масштабируемые конкурентные выделения памяти — это *raison d'être* (смысл существования) распределителя памяти `jemalloc`!

Взаимное исключение является наиболее очевидным барьером для параллельного ускорения, поскольку по определению оно заставляет некоторую часть вашей программы выполняться последовательно. Даже если вы добьётесь идеального масштабирования остальной части вашей программы с количеством ядер, общее ускорение, которое вы можете достичь,

ограничено длиной взаимно исключаемой, последовательной секции. Будьте внимательны к вашим взаимно исключаемым секциям и стремитесь ограничивать их только теми местами, где это строго необходимо.

ПРИМЕЧАНИЕ Для тех, кто склонен к теории: пределы достижимого ускорения в результате взаимно исключаемых секций кода можно вычислить с помощью закона Амдала (Amdahl's law).

Исчерпание разделяемого ресурса (Shared Resource Exhaustion)

К сожалению, даже если вы достигнете идеальной конкурентности внутри ваших задач, среда, с которой этим задачам нужно взаимодействовать, может сама по себе не быть идеально масштабируемой. Ядро может обрабатывать лишь определённое количество отправок на данном TCP-сокете в секунду, шина памяти может выполнять лишь определённое количество чтений за раз, а ваш GPU имеет ограниченную ёмкость для конкурентности. От этого нет лекарства. Среда — это обычно то место, где идеальная масштабируемость разваливается на практике, а исправления для таких случаев, как правило, требуют существенного переинжиниринга (или даже нового оборудования!), так что в этой главе мы не будем много говорить об этой теме. Просто помните, что масштабируемость редко является чем-то, чего можно «достичь», и скорее является чем-то, к чему вы лишь стремитесь.

Ложное разделение (False Sharing)

Ложное разделение возникает, когда две операции, которые не должны соперничать друг с другом, всё равно соперничают, препятствуя эффективному одновременному выполнению. Обычно это происходит из-за того, что две операции случайно пересекаются на некотором разделяемом ресурсе, хотя они используют несвязанные части этого ресурса.

Простейший пример этого — чрезмерное разделение блокировки, когда блокировка охраняет некоторое составное состояние, и двум операциям, которые в остальном независимы, обеим нужно взять эту блокировку, чтобы обновить свои конкретные части состояния. Это означает, что операциям приходится выполняться последовательно вместо параллельного. В некоторых случаях возможно разбить единую блокировку на две, по одной на каждую из непересекающихся частей, что позволяет операциям выполняться параллельно. Однако не всегда столь же просто разбить блокировку подобным

образом — состояние может разделять единую блокировку, потому что некоторой третьей операции нужно заблокировать все части состояния. Обычно вы всё же можете разбить блокировку, но тогда вам нужно быть осторожным с порядком, в котором разные потоки берут разбитые блокировки, чтобы избежать взаимных блокировок (deadlocks), которые могут возникнуть, когда две операции пытаются взять их в разном порядке (это известно как «проблема обедающих философов», если вам любопытно). Альтернативно, для некоторых задач вы можете полностью избежать критической секции, используя свободную от блокировок (lock-free) версию базового алгоритма, хотя их тоже непросто сделать правильно. В конечном счёте, ложное разделение — это трудная проблема для решения, и нет единого универсального решения, но выявление проблемы — это хорошее начало.

Более тонкий пример ложного разделения возникает на уровне CPU, как мы кратко обсуждали в главе 3. CPU внутренне оперирует памятью в терминах кэш-линий (cache lines) — более длинных последовательностей последовательных байтов в памяти, а не отдельных байтов, чтобы амортизировать стоимость обращений к памяти. Например, на большинстве процессоров Intel размер кэш-линии составляет 64 байта. Это означает, что каждая операция с памятью на самом деле в итоге читает или записывает некоторое кратное 64 байтам. Ложное разделение вступает в игру, когда два ядра хотят обновить значения двух разных байтов, которые случайно попадают в одну и ту же кэш-линию; эти обновления должны выполняться последовательно, хотя обновления логически непересекающиеся.

Это может показаться слишком низкоуровневым, чтобы иметь значение, но на практике этот вид ложного разделения может существенно уменьшить параллельное ускорение приложения. Представьте, что вы выделяете массив целочисленных значений, чтобы указывать, сколько операций завершил каждый поток, но все эти целые числа попадают в одну и ту же кэш-линию — теперь все ваши в остальном параллельные потоки будут соперничать за эту одну кэш-линию при каждой выполняемой ими операции. Если операции относительно быстрые, *большая часть* вашего времени выполнения может в итоге уходить на состязание за эти счётчики!

Хитрость для избежания ложного разделения кэш-линий состоит в том, чтобы дополнять (padding) ваши значения так, чтобы они имели размер кэш-линии.

Тогда два соседних значения всегда попадают на разные кэш-линии. Но, конечно, это также раздувает размер ваших структур данных, поэтому используйте этот подход только тогда, когда бенчмарки указывают на проблему.

СТОИМОСТЬ МАСШТАБИРУЕМОСТИ (THE COST OF SCALABILITY) Несколько ортогональный аспект конкурентности, о котором вам следует помнить, — это стоимость введения конкурентности в первую очередь. Компиляторы действительно хороши в оптимизации однопоточного кода — в конце концов, они занимаются этим уже долгое время, — и однопоточный код, как правило, обходится гораздо меньшим количеством дорогостоящих мер предосторожности (таких как блокировки, каналы или атомарные инструкции), чем конкурентный код. В совокупности различные затраты на конкурентность могут сделать параллельную программу медленнее, чем её однопоточный аналог, при *любом количестве* ядер! Вот почему важно измерять как до, так и после того, как вы оптимизируете и распараллеливаете: результаты могут вас удивить.

Если вам интересна эта тема, я настоятельно рекомендую прочитать статью Фрэнка Макшерри 2015 года «Scalability! But at what COST?» (<https://www.frank-mcsherry.org/assets/COST.pdf>), которая раскрывает некоторые особенно вопиющие примеры «дорогостоящего масштабирования».

Паттерны конкурентности (Concurrency Patterns)

В Rust есть три паттерна добавления конкурентности в ваши программы, с которыми вы будете сталкиваться довольно часто: конкурентность с разделяемой памятью, пулы рабочих и акторы. Подробное рассмотрение каждого способа, которым вы могли бы добавить конкурентность, заняло бы отдельную книгу, поэтому здесь я сосредоточусь только на этих трёх паттернах.

Разделяемая память (Shared Memory)

Конкурентность с разделяемой памятью концептуально очень проста: потоки взаимодействуют, оперируя областями памяти, разделяемыми между ними.

Это может принимать форму состояния, охраняемого мьютексом, или хранимого в хеш-карте с поддержкой конкурентного доступа из многих потоков. Множество потоков могут выполнять одну и ту же задачу над непересекающимися фрагментами данных, например, когда множество потоков выполняют некоторую функцию над непересекающимися поддиапазонами `Vec`, или же они могут выполнять разные задачи, требующие некоторого разделяемого состояния, например, в базе данных, где один поток обрабатывает пользовательские запросы к таблице, в то время как другой оптимизирует структуры данных, используемые для хранения этой таблицы, в фоновом режиме.

При использовании конкурентности с разделяемой памятью ваш выбор структур данных значителен, особенно если задействованные потоки должны тесно взаимодействовать. Обычный мьютекс может препятствовать масштабированию за пределы очень небольшого количества ядер, блокировка чтения/записи (`reader/writer lock`) может позволить гораздо больше конкурентных чтений ценой более медленных записей, а шардированная блокировка чтения/записи может позволить идеально масштабируемые чтения ценой того, что записи станут крайне разрушительными. Аналогично, некоторые конкурентные хеш-карты нацелены на хорошую всестороннюю производительность, тогда как другие специально нацелены, скажем, на конкурентные чтения там, где записи редки. В общем, при конкурентности с разделяемой памятью вы хотите использовать структуры данных, специально спроектированные настолько близко к вашему целевому сценарию использования, насколько это возможно, чтобы вы могли воспользоваться оптимизациями, которые жертвуют теми аспектами производительности, которые вашему приложению безразличны, ради тех, которые ему важны.

Конкурентность с разделяемой памятью хорошо подходит для сценариев использования, где потокам нужно совместно обновлять некоторое разделяемое состояние способом, который не коммутирует. То есть, если один поток должен обновить состояние `s` некоторой функцией `f`, а другой должен обновить состояние функцией `g`, и $f(g(s)) \neq g(f(s))$, то конкурентность с разделяемой памятью, вероятно, необходима. Если же это не так, то два других паттерна, вероятно, подойдут лучше, поскольку они, как правило, ведут к более простым и более производительным проектам.

ПРИМЕЧАНИЕ Для некоторых задач есть известные алгоритмы,

которые могут обеспечить конкурентные операции с разделяемой памятью без использования блокировок. По мере роста количества ядер эти свободные от блокировок (lock-free) алгоритмы могут масштабироваться лучше, чем алгоритмы на основе блокировок, хотя они также часто имеют более низкую производительность на ядро из-за своей сложности. Как всегда в вопросах производительности, сначала проводите бенчмарки, а затем ищите альтернативные решения.

Пулы рабочих (Worker Pools)

В модели пула рабочих множество идентичных потоков получают задания из разделяемой очереди заданий, которые они затем выполняют полностью независимо. Веб-серверы, например, часто имеют пул рабочих, обрабатывающий входящие соединения, а многопоточные исполнительные среды для асинхронного кода, как правило, используют пул рабочих для коллективного выполнения всех футур приложения (или, точнее, его задач верхнего уровня).

Границы между конкурентностью с разделяемой памятью и пулами рабочих часто размыты, поскольку пулы рабочих, как правило, используют конкурентность с разделяемой памятью для координации того, как они берут задания из очереди и как возвращают незавершённые задания обратно в очередь. Например, допустим, вы используете библиотеку для параллелизма данных `gaup`, чтобы выполнить некоторую функцию над каждым элементом вектора параллельно. За кулисами `gaup` разворачивает пул рабочих, разбивает вектор на поддиапазоны, а затем раздаёт поддиапазоны потокам в пуле. Когда поток в пуле заканчивает диапазон, `gaup` устривает так, чтобы он начал работать над следующим необработанным поддиапазоном. Вектор разделяется между всеми рабочими потоками, и потоки координируются через структуру данных типа очереди с разделяемой памятью, которая поддерживает похищение работы (work stealing).

Похищение работы — это ключевая особенность большинства пулов рабочих. Основная предпосылка состоит в том, что если один поток заканчивает свою работу рано и больше нет доступной неназначенной работы, этот поток может похитить задания, которые уже были назначены другому рабочему потоку, но ещё не были начаты. Не все задания занимают одинаковое количество

времени на завершение, поэтому даже если каждому рабочему дано одинаковое *количество* заданий, некоторые рабочие могут закончить свои задания быстрее, чем другие. Вместо того чтобы простаивать и ждать, пока завершатся потоки, которым достались более долгоиграющие задания, те потоки, что заканчивают рано, должны помочь отстающим, чтобы общая операция завершилась быстрее.

Это весьма непростая задача — реализовать структуру данных, которая поддерживает такого рода похищение работы без значительных накладных расходов от того, что потоки постоянно пытаются похитить работу друг у друга (и обычно она включает конкурентность с разделяемой памятью), но эта особенность жизненно важна для высокопроизводительного пула рабочих. Из-за связанной с этим сложности многие пулы рабочих начинают с использования простого канала с несколькими производителями и несколькими потребителями (*multiproducer, multiconsumer*) для отправки работы рабочим потокам. Однако со временем это, как правило, становится узким местом. Если вы обнаружите, что вам нужен пул рабочих, лучше всего обычно использовать тот, в который уже вложено много работы, или хотя бы повторно использовать структуры данных из существующего, а не писать собственный с нуля.

Пулы рабочих хорошо подходят, когда работа, которую выполняет каждый поток, одинакова, но данные, над которыми она выполняется, различаются. В операции параллельного отображения (*map*) *gauron* каждый поток выполняет одно и то же вычисление отображения; они просто выполняют его над разными подмножествами базовых данных. В многопоточной асинхронной исполнительной среде каждый поток просто вызывает `Future::poll`; они просто вызывают его на разных футурах. Если вы начинаете нуждаться в различении между потоками в вашем пуле потоков, то, вероятно, более уместен иной проект.

ПУЛЫ СОЕДИНЕНИЙ (CONNECTION POOLS) Пул соединений — это конструкция с разделяемой памятью, которая хранит набор установленных соединений и раздаёт их потокам, которым нужно соединение. Это распространённый паттерн проектирования в библиотеках, которые управляют соединениями с внешними сервисами. Если потоку нужно соединение, но ни одного не доступно, то либо устанавливается новое соединение, либо поток вынужден

заблокироваться. Когда поток закончил с соединением, он возвращает это соединение в пул и тем самым делает его доступным другим потокам, которые могут ожидать.

Обычно самая трудная задача для пула соединений — управление жизненными циклами соединений. Соединение может быть возвращено в пул в каком бы состоянии оно ни было оставлено последним использовавшим его потоком. Поэтому пул соединений должен убедиться, что любое состояние, связанное с соединением, будь то на клиенте или на сервере, было сброшено, так чтобы, когда соединение впоследствии используется другим потоком, этот поток мог действовать так, как будто ему дали свежее, выделенное соединение.

Актеры (Actors)

Модель конкурентности на акторах во многих отношениях является противоположностью модели пула рабочих. В то время как пул рабочих имеет множество идентичных потоков, разделяющих очередь заданий, в модели акторов есть множество отдельных очередей заданий, по одной для каждой «темы» заданий. Каждая очередь заданий питает определённого актора, который обрабатывает все задания, относящиеся к подмножеству состояния приложения. Этим состоянием может быть соединение с базой данных, файл, структура данных для сбора метрик или любая другая структура, к которой, как вы можете представить, многим потокам может понадобиться доступ. Что бы это ни было, единственный актер владеет этим состоянием, и если некоторая задача хочет взаимодействовать с этим состоянием, ей нужно отправить сообщение владеющему актору, резюмирующее операцию, которую она хочет выполнить. Когда владеющий актер получает это сообщение, он выполняет указанное действие и отвечает запрашивающей задаче результатом операции, если это уместно. Поскольку актер имеет эксклюзивный доступ к своему внутреннему ресурсу, никакие блокировки или другие механизмы синхронизации не требуются.

Ключевой момент в паттерне акторов состоит в том, что все акторы общаются друг с другом. Если, скажем, актер отвечает за логирование, которому нужно писать в файл и в таблицу базы данных, он мог бы отправить сообщения актерам, отвечающим за каждое из этих действий, прося их выполнить

соответствующие действия, а затем перейти к следующему событию логирования. Таким образом, модель акторов скорее напоминает паутину, чем спицы колеса — пользовательский запрос к веб-серверу может начаться как единственный запрос к актору, ответственному за это соединение, но затем транзитивно породить десятки, сотни или даже тысячи сообщений к акторам глубже в системе, прежде чем пользовательский запрос будет удовлетворён.

Ничто в модели акторов не требует, чтобы каждый актор был собственным потоком. Напротив, большинство акторных систем предлагают, чтобы было большое количество акторов, и поэтому актор должен отображаться на задачу, а не на поток. В конце концов, акторам нужен эксклюзивный доступ к их обёрнутым ресурсам только когда они выполняются, и им безразлично, находятся ли они на собственном потоке или нет. На самом деле очень часто модель акторов используется в сочетании с моделью пула рабочих — например, приложение, использующее многопоточную асинхронную исполнительную среду Tokio, может породить асинхронную задачу для каждого актора, и Tokio затем сделает выполнение каждого актора заданием в своём пуле рабочих. Таким образом, выполнение данного актора может перемещаться от потока к потоку в пуле рабочих по мере того, как актор уступает и возобновляется, но каждый раз, когда актор выполняется, он сохраняет эксклюзивный доступ к своему обёрнутому ресурсу.

Модель конкурентности на акторах хорошо подходит для случаев, когда у вас есть множество ресурсов, которые могут работать относительно независимо, и где есть мало или нет возможности для конкурентности внутри каждого ресурса. Например, у операционной системы может быть актор, ответственный за каждое аппаратное устройство, а у веб-сервера может быть актор для каждого внутреннего (backend) соединения с базой данных. Модель акторов работает не так хорошо, если вам нужно лишь несколько акторов, если работа значительно перекошена между акторами, или если некоторые акторы становятся слишком крупными — во всех этих случаях ваше приложение может в итоге упереться в узкое место скорости выполнения единственного актора в системе. Поскольку каждый актор ожидает иметь эксклюзивный доступ к своему маленькому кусочку мира, вы не можете распараллелить выполнение того одного актора, ставшего узким местом.

Асинхронность и параллелизм (Asynchrony and Parallelism)

Как мы обсуждали в главе 9, асинхронность в Rust обеспечивает конкурентность без параллелизма — мы можем использовать такие конструкции, как `select` и `join`, чтобы один поток опрашивал несколько футур и продолжал, когда одна, некоторые или все из них завершатся. Поскольку никакого параллелизма не задействовано, конкурентность с футурами в принципе не требует, чтобы эти футуры были `Send`. Даже порождение футуры для выполнения в качестве дополнительной задачи верхнего уровня в принципе не требует `Send`, поскольку единственный поток исполнителя может управлять опросом множества футур сразу.

Однако в *большинстве* случаев приложения хотят и конкурентность, и параллелизм. Например, если веб-приложение конструирует футуру для каждого входящего соединения и потому имеет множество активных соединений сразу, оно, вероятно, хочет, чтобы асинхронный исполнитель мог использовать более одного ядра на хост-компьютере. Это не произойдёт само собой: ваш код должен явно сообщить исполнителю, какие футуры могут выполняться параллельно, а какие нет.

В частности, исполнителю должны быть даны две части информации, чтобы он знал, что может распределить работу в футурах по пулу рабочих потоков. Первая состоит в том, что рассматриваемые футуры являются `Send` — если они таковыми не являются, исполнителю не разрешено отправлять футуры другим потокам для обработки, и никакой параллелизм невозможен; только поток, сконструировавший такие футуры, может их опрашивать.

Вторая часть информации — это то, как разбить футуры на задачи, которые могут работать независимо. Это связано с обсуждением задач против футур из главы 9: если одна гигантская `Future` содержит ряд экземпляров `Future`, которые сами соответствуют задачам, способным работать параллельно, исполнитель всё равно должен вызывать `poll` на `Future` верхнего уровня, и он должен делать это из единственного потока, поскольку `poll` требует `&mut self`. Таким образом, чтобы достичь параллелизма с футурами, вам нужно явно породить футуры, которые вы хотите запускать параллельно. Кроме того, из-за первого требования функция исполнителя, которую вы используете для этого, потребует, чтобы передаваемая `Future` была `Send`.

АСИНХРОННЫЕ ПРИМИТИВЫ СИНХРОНИЗАЦИИ (ASYNCHRONOUS SYNCHRONIZATION PRIMITIVES)

Большинство примитивов синхронизации, которые существуют для блокирующего кода (вспомните `std::sync`), также имеют асинхронные аналоги. Существуют асинхронные варианты каналов, мьютексов, блокировок чтения/записи, барьеров и всевозможных других подобных конструкций. Они нам нужны, поскольку, как обсуждалось в главе 9, блокировка внутри футуры задержит другую работу, которую исполнителю может понадобиться выполнить, и потому нежелательна.

Однако асинхронные версии этих примитивов часто медленнее, чем их синхронные аналоги, из-за дополнительного механизма, необходимого для выполнения нужных пробуждений. По этой причине вы можете захотеть использовать синхронные примитивы синхронизации даже в асинхронных контекстах, всякий раз когда их использование не рискует заблокировать исполнитель. Например, хотя в целом верно, что захват `Mutex` может заблокировать на долгое время, это может быть не так для конкретного `Mutex`, который, возможно, захватывается лишь редко и только на короткие периоды времени. В этом случае блокировка на короткое время, пока `Mutex` снова не станет доступным, скорее всего, не вызовет никаких проблем. Вам следует убедиться, что вы никогда не уступаете и не выполняете другие долгоиграющие операции, удерживая `MutexGuard`, но за исключением этого вы не должны столкнуться с проблемами.

Однако, как всегда с такими оптимизациями, убедитесь, что сначала измеряете, и выбирайте синхронный примитив только если он даёт вам значительные улучшения производительности. Если нет, то дополнительные «ружья, стреляющие в ногу», вводимые использованием синхронного примитива в асинхронном контексте, вероятно, того не стоят.

Низкоуровневая конкурентность (Lower-Level Concurrency)

Стандартная библиотека предоставляет модуль `std::sync::atomic`, который даёт доступ к базовым примитивам CPU, на которых строятся более высокоуровневые конструкции, такие как каналы и мьютексы. Эти примитивы

приходят в форме атомарных типов с именами, начинающимися с `Atomic`, — `AtomicUsize`, `AtomicI32`, `AtomicBool`, `AtomicPtr` и так далее, — типа `Ordering` и двух функций, называемых `fence` и `compiler_fence`. Мы рассмотрим каждый из них в следующих нескольких разделах.

Эти типы — те блоки, которые используются для построения любого кода, которому нужно взаимодействовать между потоками. Мьютексы, каналы, барьеры, конкурентные хеш-таблицы, стеки без блокировок и все остальные конструкции синхронизации в конечном счёте полагаются на эти примитивы для выполнения своей работы. Они также пригодятся сами по себе для лёгкого взаимодействия между потоками, где тяжеловесная синхронизация вроде мьютекса избыточна — например, чтобы инкрементировать разделяемый счётчик или установить разделяемое булево значение в `true`.

Атомарные типы особенны тем, что они имеют определённую семантику для того, что происходит, когда несколько потоков пытаются обратиться к ним конкурентно. Все эти типы поддерживают (в основном) один и тот же API: `load`, `store`, `fetch_*` и `compare_exchange`. В остальной части этого раздела мы рассмотрим, что они делают, как правильно их использовать и для чего они полезны. Но сначала нам нужно поговорить о низкоуровневых операциях с памятью и упорядочении памяти.

Операции с памятью (Memory Operations)

Неформально мы часто говорим об обращении к переменным как о «чтении из» или «записи в» память. В реальности между кодом, который использует переменную, и фактическими инструкциями CPU, которые обращаются к вашему оборудованию памяти, есть много механизмов. Важно понимать эти механизмы, по крайней мере на высоком уровне, чтобы понять, как ведут себя конкурентные обращения к памяти.

Компилятор решает, какие инструкции выпускать, когда ваша программа читает значение переменной или присваивает ей новое. Ему разрешено выполнять всевозможные преобразования и оптимизации вашего кода, и он может в итоге переупорядочить операторы вашей программы, устранить операции, которые он считает избыточными, или использовать регистры CPU вместо реальной памяти для хранения промежуточных вычислений. Компилятор подчиняется ряду ограничений на эти преобразования, но в конечном счёте лишь подмножество ваших обращений к переменным

фактически превращается в инструкции доступа к памяти.

На уровне CPU инструкции памяти бывают двух основных форм: загрузки (loads) и сохранения (stores). Загрузка извлекает байты из ячейки в памяти в регистр CPU, а сохранение помещает байты из регистра CPU в ячейку в памяти. Загрузки и сохранения оперируют небольшими порциями памяти за раз: обычно 8 байтами или меньше на современных CPU. Если обращение к переменной охватывает больше байтов, чем может быть доступно одной загрузкой или сохранением, компилятор автоматически превращает его в несколько инструкций загрузки или сохранения, по необходимости. У CPU есть некоторая свобода в том, как он выполняет инструкции программы, чтобы лучше использовать оборудование и улучшить производительность программы. Например, современные CPU часто выполняют инструкции параллельно или даже не по порядку, когда у них нет зависимостей друг от друга. Между каждым CPU и DRAM вашего компьютера также есть несколько слоёв кэшей, что означает, что загрузка данной ячейки памяти может не обязательно увидеть последнее сохранение в эту ячейку памяти, если судить по реальному времени.

В большинстве кода компилятору и CPU разрешено преобразовывать код только способами, которые не влияют на семантику результирующей программы, так что эти преобразования невидимы для программиста. Однако в контексте параллельного выполнения эти преобразования могут оказать значительное влияние на поведение приложения. Поэтому CPU обычно предоставляют несколько различных вариантов инструкций загрузки и сохранения, каждый с разными гарантиями относительно того, как CPU может их переупорядочивать и как они могут чередоваться с параллельными операциями на других CPU. Аналогично, компиляторы (или, скорее, язык, который компилирует компилятор) предоставляют различные аннотации, которые вы можете использовать, чтобы навязать определённые ограничения выполнения для некоторого подмножества ваших обращений к памяти. В Rust эти аннотации приходят в форме атомарных типов и их методов, на разбор которых мы потратим остальную часть этого раздела.

Атомарные типы (Atomic Types)

Атомарные типы Rust называются так потому, что к ним можно обращаться атомарно — то есть значение переменной атомарного типа записывается всё

сразу и никогда не будет записываться с использованием нескольких сохранений, что гарантирует, что загрузка такой переменной не сможет наблюдать ситуацию, когда лишь некоторые из байтов, составляющих значение, изменились, в то время как другие (пока) нет. Это легче всего понять путём контраста с неатомарными типами. Например, переприсваивание нового значения кортежу типа `(i64, i64)` обычно требует двух инструкций сохранения CPU, по одной на каждое 8-байтовое значение. Если бы один поток выполнял оба этих сохранения, другой поток мог бы (если мы на секунду проигнорируем проверку заимствований) прочитать значение кортежа после первого сохранения, но до второго, и таким образом в итоге получить несогласованное представление о значении кортежа. Он в итоге прочитал бы новое значение для первого элемента и старое значение для второго элемента — значение, которое в действительности никогда не сохранял ни один поток.

CPU может атомарно обращаться к значениям только определённых размеров, поэтому существует лишь несколько атомарных типов, и все они живут в модуле `atomic`. Каждый атомарный тип имеет один из тех размеров, для которых CPU поддерживает атомарный доступ, с несколькими вариациями для таких вещей, как является ли значение знаковым, и для различения между атомарным `usize` и указателем (который имеет тот же размер, что и `usize`). Кроме того, атомарные типы имеют явные методы для загрузки и сохранения значений, которые они хранят, и горстку более сложных методов, к которым мы вернёмся позже, так что отображение между кодом, который пишет программист, и результирующими инструкциями CPU яснее. Например, `AtomicI32::load` выполняет одну загрузку знакового 32-битного значения, а `AtomicPtr::store` выполняет одно сохранение значения размером с указатель (64 бита на 64-битной платформе).

Упорядочение памяти (Memory Ordering)

Большинство методов атомарных типов принимают аргумент типа `Ordering`, который диктует ограничения упорядочения памяти, которым подчиняется атомарная операция. Между разными потоками загрузки и сохранения атомарного значения могут быть упорядочены компилятором и CPU только в таких чередованиях, которые совместимы с запрошенным упорядочением памяти каждой из атомарных операций над этим атомарным значением. В следующих нескольких разделах мы увидим несколько примеров того, почему

важно и необходимо контролировать упорядочение, чтобы получить ожидаемую семантику от компилятора и CPU.

Упорядочение памяти часто кажется контринтуитивным, поскольку мы, люди, любим читать программы сверху вниз и воображать, что они выполняются строка за строкой, — но это не то, как код в действительности выполняется, когда он попадает на оборудование. Обращения к памяти могут быть переупорядочены или даже полностью исключены, а записи в одном потоке могут не быть немедленно видны другим потокам, даже если более поздние записи в порядке программы уже были замечены.

Представьте это так: каждая ячейка памяти видит последовательность модификаций, приходящих от разных потоков, и последовательности модификаций для разных ячеек памяти независимы. Если два потока T1 и T2 оба пишут в ячейку памяти M, то даже если T1 выполнен первым по измерению пользователя с секундомером, запись T2 в M может всё равно казаться произошедшей первой для M, при отсутствии каких-либо других ограничений между выполнением двух потоков. По существу, *компьютер не принимает во внимание реальное время*, когда определяет значение данной ячейки памяти, — важны только ограничения выполнения, которые программист устанавливает на то, что составляет допустимое выполнение. Например, если T1 пишет в M, а затем порождает поток T2, который затем пишет в M, компьютер должен признать запись T1 как произошедшую первой, поскольку существование T2 зависит от T1.

Если это трудно отследить, не переживайте — упорядочение памяти может выворачивать мозг, а спецификации языков, как правило, используют очень точные, но не очень интуитивные формулировки для его описания. Мы можем построить мысленную модель, которую легче ухватить, если несколько упростить, сосредоточившись вместо этого на базовой аппаратной архитектуре. По существу, память вашего компьютера структурирована как иерархия хранилищ, где листья — это регистры CPU, а корни — это хранилище на ваших физических чипах памяти, часто называемое основной памятью (main memory). Между этими двумя есть несколько слоёв кэшей, и разные слои иерархии могут располагаться на разных частях оборудования. Когда поток выполняет сохранение в ячейку памяти, в действительности происходит то, что CPU начинает запрос на запись, чтобы переместить значение в данный регистр CPU, который затем должен пробираться вверх по

иерархии памяти к основной памяти. Когда поток выполняет загрузку, запрос течёт вверх по иерархии, пока не достигнет слоя, у которого есть это значение в наличии, и возвращается оттуда. Здесь и кроется проблема: записи не видны везде, пока все кэши записанной ячейки памяти не будут обновлены, но другие CPU могут выполнять инструкции против той же ячейки памяти в то же время, и возникает странность. Упорядочение памяти, тогда, — это запрос точной семантики того, что происходит, когда несколько CPU обращаются к определённой ячейке памяти для определённой операции.

Имея это в виду, давайте взглянем на тип `Ordering`, который является первичным механизмом, посредством которого мы, как программисты, можем диктовать дополнительные ограничения на то, какие конкурентные выполнения допустимы.

`Ordering` определён как `enum` с вариантами, показанными в листинге 11-1.

```
enum Ordering {  
    Relaxed,  
    Release,  
    Acquire,  
    AcqRel,  
    SeqCst  
}
```

Листинг 11-1: Определение `Ordering`

Каждый из них накладывает разные ограничения на отображение из исходного кода в семантику выполнения, и мы исследуем каждый по очереди в остальной части этого раздела.

Расслабленное упорядочение (`Relaxed Ordering`)

Расслабленное упорядочение по существу не гарантирует ничего о конкурентном доступе к значению сверх того факта, что доступ является атомарным. В частности, расслабленное упорядочение не даёт никаких гарантий об относительном порядке обращений к памяти между разными потоками. Это самая слабая форма упорядочения памяти. Листинг 11-2 показывает простую программу, в которой два потока обращаются к двум атомарным переменным, используя `Ordering::Relaxed`.

```
static X: AtomicBool = AtomicBool::new(false);
```

```

static Y: AtomicBool = AtomicBool::new(false);

let t1 = spawn(|| {
  ❶ let r1 = Y.load(Ordering::Relaxed);
  ❷ X.store(r1, Ordering::Relaxed);
});
let t2 = spawn(|| {
  ❸ let r2 = X.load(Ordering::Relaxed);
  ❹ Y.store(true, Ordering::Relaxed)
});

```

Листинг 11-2: Два состязающихся потока с `Ordering::Relaxed`

Глядя на поток, порождённый как `t2`, вы могли бы ожидать, что `r2` никогда не сможет быть `true`, поскольку все значения равны `false`, пока тот же поток не присвоит `true` переменной `Y` на строке *после* чтения `X`. Однако при расслабленном упорядочении памяти этот исход вполне возможен. Причина в том, что CPU разрешено переупорядочивать задействованные загрузки и сохранения. Давайте пройдемся по тому, что именно происходит здесь, чтобы сделать `r2 = true` возможным.

Сначала CPU замечает, что ❹ не обязано происходить после ❸, поскольку ❹ не использует никакого вывода или побочного эффекта ❸. То есть ❹ не имеет зависимости выполнения от ❸. Поэтому CPU решает переупорядочить их, потому что по причинам *взмах руками* это сделает вашу программу быстрее. CPU, таким образом, идёт вперёд и выполняет ❹ первым, устанавливая `Y = true`, хотя ❸ ещё не выполнилось. Затем `t2` усыпляется операционной системой, и поток `t1` выполняет несколько инструкций, или `t1` просто выполняется на другом ядре. В `t1` компилятор действительно должен выполнить ❶ первым, а затем ❷, поскольку ❷ зависит от значения, прочитанного в ❶. Поэтому `t1` читает `true` из `Y` (записанное ❹) в `r1`, а затем записывает это обратно в `X`. Наконец, `t2` выполняет ❸, которое читает `X` и получает `true`, как было записано ❷.

Расслабленное упорядочение памяти допускает это выполнение, поскольку оно не накладывает никаких дополнительных ограничений на конкурентное выполнение. То есть при расслабленном упорядочении памяти компилятор должен лишь обеспечить, чтобы зависимости выполнения на любом данном потоке соблюдались (точно так же, как если бы атомарные типы не были задействованы); он не обязан давать никаких обещаний об чередовании конкурентных операций. Переупорядочение ❸ и ❹ допустимо для

однопоточного выполнения, так что оно допустимо и при расслабленном упорядочении.

В некоторых случаях такого рода переупорядочение нормально. Например, если у вас есть счётчик, который просто отслеживает метрики, не имеет особого значения, когда именно он выполняется относительно других инструкций, и `Ordering::Relaxed` подходит. В других случаях это может быть катастрофично: скажем, если ваша программа использует `r2`, чтобы выяснить, были ли уже установлены защиты безопасности, и в итоге ошибочно полагает, что они уже установлены.

Вы обычно не замечаете это переупорядочение при написании кода, который не делает причудливого использования атомарных операций, — CPU должен обещать, что не будет наблюдаемой разницы между кодом как написано и тем, что фактически выполняет каждый отдельный поток, так что всё кажется так, будто оно выполняется по порядку в точности так, как вы написали. Это называется соблюдением порядка программы (`program order`) или порядка вычисления (`evaluation order`); термины являются синонимами.

Упорядочение `Release/Acquire` (`Release/Acquire Ordering`)

На следующей ступени в иерархии упорядочения памяти у нас есть `Ordering::Acquire`, `Ordering::Release` и `Ordering::AcqRel` (`acquire plus release`, захват плюс освобождение). На высоком уровне они устанавливают зависимость выполнения между сохранением в одном потоке и загрузкой в другом, а затем ограничивают то, как операции могут быть переупорядочены относительно аннотированных загрузки и сохранения. Что особенно важно, эти зависимости не просто устанавливают отношение между сохранением и загрузкой одного значения, но также накладывают ограничения упорядочения на другие загрузки и сохранения в задействованных потоках. Это потому, что каждое выполнение должно соблюдать порядок программы; если загрузка в потоке В имеет зависимость от некоторого сохранения в потоке А (сохранение в А должно выполниться до загрузки в В), то любое чтение или запись в В после этой загрузки также должны произойти после того сохранения в А.

ПРИМЕЧАНИЕ Упорядочение памяти `Acquire` может применяться только к загрузкам, `Release` — только к сохранениям, а `AcqRel` — только к операциям, которые одновременно загружают *и* сохраняют (вроде `fetch_add`).

Конкретно, эти упорядочения памяти накладывают следующие ограничения на выполнение:

1. Загрузки и сохранения не могут быть перемещены вперёд за сохранение с `Ordering::Release`.
1. Загрузки и сохранения не могут быть перемещены назад перед загрузку с `Ordering::Acquire`.
1. Загрузка с `Ordering::Acquire` некоторой переменной должна увидеть все сохранения, которые произошли до сохранения с `Ordering::Release`, которое сохранило то, что загрузила эта загрузка.

Чтобы увидеть, как эти упорядочения памяти меняют дело, листинг 11-3 показывает листинг 11-2 снова, но с упорядочением памяти, заменённым на `Acquire` и `Release`.

```
static X: AtomicBool = AtomicBool::new(false);
static Y: AtomicBool = AtomicBool::new(false);

let t1 = spawn(|| {
    let r1 = Y.load(Ordering::Acquire);
    X.store(r1, Ordering::Release);
});
let t2 = spawn(|| {
    ❶ let r2 = X.load(Ordering::Acquire);
    ❷ Y.store(true, Ordering::Release)
});
```

Листинг 11-3: Листинг 11-2 с упорядочением памяти `Release/Acquire`

Эти дополнительные ограничения означают, что теперь больше невозможно, чтобы `t2` увидел `r2 = true`. Чтобы понять почему, рассмотрим первичную причину странного исхода в листинге 11-2: переупорядочение ❶ и ❷. Самое первое ограничение, на сохранениях с `Ordering::Release`, диктует, что мы не можем переместить ❶ ниже ❷, так что всё в порядке!

Но эти правила полезны и за пределами этого простого примера. Например, представьте, что вы реализуете блокировку взаимного исключения. Вы хотите убедиться, что любые загрузки и сохранения, которые поток выполняет, пока удерживает блокировку, выполняются только пока он действительно её удерживает, и видны любому потоку, который возьмёт блокировку позже. Это именно то, что позволяют вам сделать `Release` и `Acquire`. Выполняя сохранение

Release для освобождения блокировки и загрузку Acquire для захвата блокировки, вы можете гарантировать, что загрузки и сохранения в критической секции никогда не перемещаются до фактического захвата блокировки или после её освобождения!

ПРИМЕЧАНИЕ На некоторых архитектурах CPU, например x86, упорядочение Release/Acquire гарантируется оборудованием, и нет дополнительной стоимости использования Ordering::Release и Ordering::Acquire вместо Ordering::Relaxed. На других архитектурах это не так, и ваша программа может получить ускорение, если вы переключитесь на Relaxed для атомарных операций, которые могут терпеть более слабые гарантии упорядочения памяти.

Последовательно согласованное упорядочение (Sequentially Consistent Ordering)

Последовательно согласованное упорядочение (Ordering::SeqCst) — это самое сильное упорядочение памяти, к которому у нас есть доступ. Его точные гарантии несколько трудно зафиксировать, но очень в общих чертах оно требует не только того, чтобы каждый поток видел результаты, согласованные с Release/Acquire, но также чтобы все потоки видели *одно и то же* упорядочение друг с другом. Это лучше всего видно путём контраста с поведением Acquire и Release. В частности, упорядочение Release/Acquire *не* гарантирует, что если два потока А и В атомарно загружают значения, записанные двумя другими потоками X и Y, то А и В увидят согласованный паттерн того, когда X писал относительно Y. Это довольно абстрактно, поэтому рассмотрим пример в листинге 11-4, который показывает случай, где упорядочение Release/Acquire может дать неожиданные результаты. После этого мы увидим, как последовательно согласованное упорядочение избегает этого конкретного неожиданного исхода.

```
static X: AtomicBool = AtomicBool::new(false);
static Y: AtomicBool = AtomicBool::new(false);
static Z: AtomicI32 = AtomicI32::new(0);

let t1 = spawn(|| {
    X.store(true, Ordering::Release);
});
let t2 = spawn(|| {
    Y.store(true, Ordering::Release);
});
```



```

let t3 = spawn(|| {
  while (!X.load(Ordering::Acquire)) {}
  ❶ if (Y.load(Ordering::Acquire)) {
    Z.fetch_add(1, Ordering::Relaxed); }
});
let t4 = spawn(|| {
  while (!Y.load(Ordering::Acquire)) {}
  ❷ if (X.load(Ordering::Acquire)) {
    Z.fetch_add(1, Ordering::Relaxed); }
});

```

Листинг 11-4: Странные результаты с упорядочением Release/Acquire

Два потока `t1` и `t2` устанавливают `X` и `Y` в `true` соответственно. Поток `t3` ждёт, пока `X` не станет `true`; как только `X` равно `true`, он проверяет, равно ли `Y` тоже `true`, и если да, прибавляет 1 к `Z`. Поток `t4` вместо этого ждёт, пока `Y` не станет `true`, а затем проверяет, равно ли `X` тоже `true`, и если да, прибавляет 1 к `Z`. На этом этапе вопрос таков: каковы возможные значения `Z` после того, как все потоки завершатся? Прежде чем я покажу вам ответ, попробуйте проработать это самостоятельно, учитывая определения упорядочения Release и Acquire из предыдущего раздела.

Сначала давайте подытожим условия, при которых `Z` инкрементируется. Поток `t3` инкрементирует `Z`, если он видит, что `Y` равно `true` после того, как он наблюдает, что `X` равно `true`, что может произойти только если `t2` выполняется раньше, чем `t3` вычислит загрузку в ❶. И наоборот, `t4` инкрементирует `Z`, если он видит, что `X` равно `true` после того, как он наблюдает, что `Y` равно `true`, так что только если `t1` выполняется раньше, чем `t4` вычислит загрузку в ❷. Чтобы упростить объяснение, давайте пока предположим, что каждый поток выполняется до завершения, как только он запускается.

Логически, тогда, `Z` может быть инкрементировано дважды, если потоки выполняются в порядке 1, 2, 3, 4 — и `X`, и `Y` установлены в `true`, а затем `t3` и `t4` выполняются и обнаруживают, что их условия для инкремента `Z` выполнены. Аналогично, `Z` может быть инкрементировано лишь один раз, если потоки выполняются в порядке 1, 3, 2, 4. Это удовлетворяет условию `t4` для инкремента `Z`, но не условию `t3`. Однако получить `Z` равным 0 *кажется* невозможным: если мы хотим помешать `t3` инкрементировать `Z`, то `t2` должен выполняться после `t3`. Поскольку `t3` выполняется только после `t1`, это подразумевает, что `t2` выполняется после `t1`. Однако `t4` не выполнится, пока не выполнится `t2`, так что `t1` должен был выполниться и установить `X` в `true` к

моменту, когда `t4` выполняется, и поэтому `t4` инкрементирует `Z`.

Наша неспособность получить `Z` равным `0` проистекает в основном из нашей человеческой склонности к линейным объяснениям; это произошло, затем то произошло, затем это произошло. Компьютеры не ограничены тем же способом, и им незачем упаковывать все события в единый глобальный порядок. В правилах для `Release` и `Acquire` нет ничего, что говорило бы, что `t3` должен наблюдать тот же порядок выполнения для `t1` и `t2`, что и `t4`. Что касается компьютера, нормально позволить `t3` наблюдать, что `t1` выполнен первым, в то время как `t4` наблюдает, что `t2` выполнен первым. Имея это в виду, выполнение, в котором `t3` наблюдает, что `Y` равно `false` после того, как он наблюдает, что `X` равно `true` (подразумевая, что `t2` выполняется после `t1`), в то время как в том же выполнении `t4` наблюдает, что `X` равно `false` после того, как он наблюдает, что `Y` равно `true` (подразумевая, что `t2` выполняется до `t1`), вполне разумно, даже если это кажется возмутительным нам, простым людям.

Как мы обсуждали ранее, `Release/Acquire` требует только того, чтобы загрузка с `Ordering::Acquire` некоторой переменной видела все сохранения, которые произошли до сохранения с `Ordering::Release`, которое сохранило то, что загрузила эта загрузка. В только что обсуждённом упорядочении компьютер *действительно* поддерживает это свойство: `t3` видит `X == true` и действительно видит все сохранения от `t1`, предшествующие установке им `X = true`, — таковых нет. Он также видит `Y == false`, которое было сохранено основным потоком при запуске программы, так что нет никаких релевантных сохранений, о которых стоило бы беспокоиться. Аналогично, `t4` видит `Y == true` и также видит все сохранения от `t2`, предшествующие установке `Y = true`, — опять же, таковых нет. Он также видит `X == false`, которое было сохранено основным потоком и не имеет предшествующего сохранения. Никакие правила не нарушены, и всё же это просто кажется как-то неправильным.

Наше интуитивное ожидание состояло в том, что мы можем поместить потоки в некий глобальный порядок, чтобы осмыслить то, что видел и делал каждый поток, но это не было справедливо для упорядочения `Release/Acquire` в этом примере. Чтобы достичь чего-то более близкого к этому интуитивному ожиданию, нам нужна последовательная согласованность. Последовательная согласованность требует, чтобы все потоки, участвующие в атомарной операции, координировались, обеспечивая, что то, что наблюдает каждый поток, соответствует (или хотя бы кажется соответствующим) *некоторому*

единому, общему порядку выполнения. Это облегчает рассуждение, но также обходится дорого.

Атомарные загрузки и сохранения, помеченные `Ordering::SeqCst`, инструктируют компилятор принять любые дополнительные меры предосторожности (например, использовать специальные инструкции CPU), необходимые, чтобы гарантировать последовательную согласованность для этих загрузок и сохранений. Точная формализация вокруг этого довольно запутанна, но последовательная согласованность по существу обеспечивает, что если бы вы посмотрели на все связанные операции `SeqCst` со всех ваших потоков, вы могли бы поместить выполнения потоков в *некий* порядок так, чтобы значения, которые были загружены и сохранены, все сходились.

Если бы мы заменили все аргументы упорядочения памяти в листинге 11-4 на `SeqCst`, было бы невозможно, чтобы `Z` равнялось 0 после того, как все потоки завершились, в точности как мы изначально ожидали. При последовательной согласованности должна быть возможность либо сказать, что `t1` определённо выполнен до `t2`, либо что `t2` определённо выполнен до `t1`, так что выполнение, где `t3` и `t4` видят разные порядки, не допускается, и потому `Z` не может быть 0.

Сравнение и обмен (Compare and Exchange)

В дополнение к `load` и `store` все атомарные типы Rust предоставляют метод под названием `compare_exchange`. Этот метод используется для атомарной *и условной* замены значения. Вы предоставляете `compare_exchange` последнее значение, которое вы наблюдали для атомарной переменной, и новое значение, которым вы хотите заменить исходное значение, и он заменит значение, только если оно по-прежнему такое же, каким было, когда вы в последний раз его наблюдали. Чтобы понять, почему это важно, взгляните на (сломанную) реализацию блокировки взаимного исключения в листинге 11-5. Эта реализация отслеживает, удерживается ли блокировка, в статической атомарной переменной `LOCK`. Мы используем булево значение `true`, чтобы представить, что блокировка удерживается. Чтобы захватить блокировку, поток ждёт, пока `LOCK` не станет `false`, затем снова устанавливает его в `true`; затем он входит в свою критическую секцию и устанавливает `LOCK` в `false`, когда его работа (`f`) завершена.

```

static LOCK: AtomicBool = AtomicBool::new(false);

fn mutex(f: impl FnOnce()) {
    // Ждём, пока блокировка не станет свободной (false).
    while LOCK.load(Ordering::Acquire)
        { /* .. TODO: избегать активного ожидания .. */ }
    // Сохраняем тот факт, что мы удерживаем блокировку.
    LOCK.store(true, Ordering::Release);
    // Вызываем f, удерживая блокировку.
    f();
    // Освобождаем блокировку.
    LOCK.store(false, Ordering::Release);
}

```

Листинг 11-5: Некорректная реализация блокировки взаимного исключения

Это в основном работает, но имеет ужасный изъян — два потока могут оба увидеть `LOCK == false` в одно и то же время, и оба выйти из цикла `while`. Затем они оба устанавливают `LOCK` в `true` и оба входят в критическую секцию, что именно то, что функция `mutex` должна была предотвратить!

Проблема в листинге 11-5 в том, что есть разрыв между тем, когда мы загружаем текущее значение атомарной переменной, и тем, когда мы впоследствии его обновляем, в течение которого другой поток может успеть выполниться и прочитать или коснуться его значения. Это в точности та проблема, которую решает `compare_exchange` — он заменяет значение за атомарной переменной, *только* если его значение по-прежнему совпадает с ранее прочитанным, а иначе уведомляет вас, что значение изменилось. Листинг 11-6 показывает исправленную реализацию с использованием `compare_exchange`.

```

static LOCK: AtomicBool = AtomicBool::new(false);

fn mutex(f: impl FnOnce()) {
    // Ждём, пока блокировка не станет свободной (false).
    loop {
        let take = LOCK.compare_exchange(
            false,
            true,
            Ordering::AcqRel,
            Ordering::Relaxed
        );
        match take {
            Ok(false) => break,
            Ok(true) | Err(false) => unreachable!(),
            Err(true) => { /* .. TODO: избегать активного ожидания .. */ }
        }
    }
}

```

```
}  
// Вызываем f, удерживая блокировку.  
f();  
// Освобождаем блокировку.  
LOCK.store(false, Ordering::Release);  
}
```

Листинг 11-6: Исправленная реализация блокировки взаимного исключения

На этот раз мы используем `compare_exchange` в цикле, и он заботится одновременно о проверке того, что блокировка в данный момент не удерживается, и о сохранении `true`, чтобы взять блокировку, если уместно. Это происходит через первый и второй аргументы `compare_exchange` соответственно: в данном случае `false`, а затем `true`. Вы можете прочитать этот вызов как «Сохрани `true`, только если текущее значение `false`». Метод `compare_exchange` возвращает `Result`, который указывает, что либо значение было успешно обновлено (`Ok`), либо что его не удалось обновить (`Err`). В любом случае он также возвращает текущее значение. Это не слишком полезно с `AtomicBool`, поскольку мы знаем, каким должно быть значение, если операция не удалась, но для чего-то вроде `AtomicI32` обновлённое текущее значение позволит вам быстро пересчитать, что сохранять, а затем попробовать снова без необходимости делать ещё одну загрузку.

ПРИМЕЧАНИЕ Обратите внимание, что `compare_exchange` проверяет только, совпадает ли значение с тем, что было передано как текущее значение. Если некоторый другой поток изменит значение атомарной переменной, а затем снова сбросит его обратно к исходному значению, `compare_exchange` на этой переменной всё равно завершится успешно. Это часто называют проблемой A-B-A (A-B-A problem).

В отличие от простых загрузок и сохранений, `compare_exchange` принимает *два* аргумента `Ordering`. Первый — это «упорядочение успеха» (success ordering), и он диктует, какое упорядочение памяти следует использовать для загрузки и сохранения, которые `compare_exchange` представляет в случае, когда значение было успешно обновлено. Второй — это «упорядочение неудачи» (failure ordering), и он диктует упорядочение памяти для загрузки, если загруженное значение не совпадает с ожидаемым текущим значением. Эти два упорядочения держатся отдельно, чтобы разработчик мог дать CPU свободу улучшить производительность выполнения путём переупорядочения загрузок и сохранений при неудаче, когда это уместно, но всё равно получить

корректное упорядочение при успехе. В данном случае нормально переупорядочивать загрузки и сохранения между неудачными итерациями цикла захвата блокировки, но *не* нормально переупорядочивать загрузки и сохранения внутри критической секции таким образом, чтобы они в итоге оказались вне её.

Хотя его интерфейс прост, `compare_exchange` — очень мощный примитив синхронизации, настолько, что теоретически доказано, что можно построить все остальные примитивы распределённого консенсуса, используя только `compare_exchange`! По этой причине он является рабочей лошадкой многих, если не большинства, конструкций синхронизации, когда вы действительно докапываетесь до деталей реализации.

Однако имейте в виду, что `compare_exchange` требует, чтобы единственный CPU имел эксклюзивный доступ к базовому значению, и потому является формой взаимного исключения на аппаратном уровне. Это, в свою очередь, означает, что `compare_exchange` может быстро стать узким местом масштабируемости: только один CPU может продвигаться за раз, так что есть часть вашего кода, которая не будет масштабироваться с количеством ядер. На самом деле, вероятно, всё хуже — CPU должны координироваться, чтобы обеспечить, что только один CPU преуспевает в `compare_exchange` для переменной за раз (взгляните на протокол MESI, если вам любопытно, как это работает), и эта координация растёт квадратично дороже, чем больше CPU задействовано!

COMPARE_EXCHANGE_WEAK Внимательный читатель документации заметит, что у `compare_exchange` есть подозрительно названный родственник, `compare_exchange_weak`, и задастся вопросом, в чём разница. Слабому (weak) варианту `compare_exchange` разрешено завершиться неудачей, даже если значение атомарной переменной всё же совпадает с ожидаемым значением, которое передал пользователь, тогда как сильный (strong) вариант в этом случае должен преуспеть.

Это может показаться странным — зачем позволять обмену атомарного значения завершиться неудачей, кроме как если значение изменилось? Ответ кроется в системных архитектурах, у которых нет нативной операции `compare_exchange`. Например, процессоры ARM вместо этого имеют операции *заблокированной загрузки* (*locked load*) и *условного сохранения* (*conditional store*), где условное сохранение завершится неудачей, если значение, прочитанное связанной

заблокированной загрузкой, не было записано с момента той загрузки. Стандартная библиотека Rust реализует `compare_exchange` на ARM путём вызова этой пары инструкций в цикле и возврата только тогда, когда условное сохранение преуспееет. Это делает код в листинге 11-6 излишне неэффективным — мы в итоге получаем вложенный цикл, который требует больше инструкций и труднее оптимизировать. Поскольку у нас в данном случае уже есть цикл, мы могли бы вместо этого использовать `compare_exchange_weak`, убрать `unreachable!()` на `Err(false)` и получить лучший машинный код на ARM и тот же скомпилированный код на x86!

Методы fetch (The Fetch Methods)

Методы `fetch` (`fetch_add`, `fetch_sub`, `fetch_and` и тому подобные) спроектированы, чтобы позволить более эффективное выполнение атомарных операций, которые коммутируют, — то есть операций, которые имеют осмысленную семантику независимо от порядка, в котором они выполняются. Мотивация для этого в том, что метод `compare_exchange` мощный, но также дорогой — если два потока оба хотят обновить единственную атомарную переменную, один преуспееет, а другой потерпит неудачу и должен будет повторить попытку. Если задействовано много потоков, им всем приходится опосредовать последовательный доступ к базовому значению, и будет много активного ожидания, пока потоки повторяют попытки при неудаче.

Для простых операций, которые коммутируют, вместо того чтобы терпеть неудачу и повторять только потому, что другой поток изменил значение, мы можем сообщить CPU, какую операцию выполнить над атомарной переменной. Он тогда выполнит эту операцию над тем значением, которое окажется текущим, когда CPU в итоге получит эксклюзивный доступ. Подумайте об `AtomicUsize`, который считает количество завершённых заданий пула. Если два потока завершают задание в одно и то же время, не имеет значения, какой из них первым обновит счётчик, лишь бы оба их инкремента были учтены.

Методы `fetch` реализуют этот вид коммутативных операций. Они выполняют операцию чтения и сохранения за один шаг и гарантируют, что операция сохранения была выполнена над атомарной переменной, когда она удерживала ровно то значение, которое вернул метод. Например, `AtomicUsize::fetch_add(1, Ordering::Relaxed)` никогда не завершается неудачей —

он всегда прибавляет 1 к текущему значению `AtomicUsize`, каким бы оно ни было, и возвращает значение `AtomicUsize` ровно на момент, когда была прибавлена единица этого потока.

Методы `fetch`, как правило, эффективнее, чем `compare_exchange`, поскольку они не требуют от потоков терпеть неудачу и повторять попытку, когда несколько потоков состязаются за доступ к переменной. У некоторых аппаратных архитектур даже есть специализированные реализации методов `fetch`, которые масштабируются гораздо лучше по мере роста количества задействованных CPU. Тем не менее, если достаточно много потоков попытаются оперировать одной и той же атомарной переменной, эти операции начнут замедляться и проявлять сублинейное масштабирование из-за необходимой координации. В общем, лучший способ значительно улучшить производительность конкурентного алгоритма — разделить состязаемые переменные на большее количество атомарных переменных, каждая из которых менее состязается, а не переключаться с `compare_exchange` на метод `fetch`.

ПРИМЕЧАНИЕ Метод `fetch_update` назван несколько обманчиво — за кулисами это в действительности просто цикл `compare_exchange_weak`, и потому его профиль производительности будет более тесно соответствовать профилю `compare_exchange`, чем другим методам `fetch`.

Здравая конкурентность (Sane Concurrency)

Писать корректный и производительный конкурентный код труднее, чем писать последовательный код; вам приходится учитывать не только возможные чередования выполнения, но и то, как ваш код взаимодействует с компилятором, CPU и подсистемой памяти. С таким широким набором «ружей, стреляющих в ногу», в вашем распоряжении легко почувствовать желание вскинуть руки и просто полностью отказаться от конкурентности. В этом разделе мы исследуем некоторые приёмы и инструменты, которые могут помочь обеспечить, чтобы вы писали корректный конкурентный код без (такого большого) страха.

Начинайте с простого (Start Simple)

Это факт жизни: простой, прямолинейный, легко отслеживаемый код с большей вероятностью будет корректным. Этот принцип применяется и к

конкурентному коду — всегда начинайте с самого простого проекта конкурентности, который вы можете придумать, затем измеряйте, и только если измерение выявит проблему с производительностью, оптимизируйте ваш алгоритм.

Чтобы следовать этому совету на практике, начинайте с паттернов конкурентности, которые не требуют замысловатого использования атомарных операций или множества мелкозернистых блокировок. Начните с нескольких потоков, которые выполняют последовательный код и взаимодействуют по каналам или которые сотрудничают через блокировки, а затем сделайте бенчмарк результирующей производительности на той рабочей нагрузке, которая вас интересует. Вы гораздо менее вероятно совершите ошибки этим способом, чем реализуя причудливые алгоритмы без блокировок или разбивая ваши блокировки на тысячу кусков, чтобы избежать ложного разделения. Для многих сценариев использования эти проекты вполне достаточно быстры; оказывается, в то, чтобы каналы и блокировки работали хорошо, вложено много времени и усилий! И если простой подход достаточно быстр для вашего сценария, зачем вводить более сложный и подверженный ошибкам код?

Если ваши бенчмарки указывают, что есть проблема с производительностью, то выясните, какая именно часть вашей системы плохо масштабируется. Сосредоточьтесь на исправлении этого узкого места изолированно, где можете, и постарайтесь сделать это с помощью малых корректировок, где это возможно. Может быть, достаточно разбить блокировку на две, а не переходить к конкурентной хеш-таблице, или ввести ещё один поток и канал вместо реализации очереди с похищением работы без блокировок. Если так, то так и сделайте.

Даже когда вам всё же приходится работать напрямую с атомарными операциями и тому подобным, держите вещи простыми, пока у вас не появится доказанная необходимость оптимизировать, — используйте сначала `Ordering::SeqCst` и `compare_exchange`, а затем итерируйте, если найдёте конкретные доказательства того, что они становятся узкими местами, с которыми нужно разобраться.

Пишите стресс-тесты (Write Stress Tests)

Как автор, вы имеете много понимания того, где могут скрываться баги в

вашем коде, не обязательно зная, что это за баги (пока, во всяком случае). Написание стресс-тестов — хороший способ вытряхнуть некоторые из скрытых багов. Стресс-тесты не обязательно выполняют сложную последовательность шагов, но вместо этого имеют множество потоков, выполняющих относительно простые операции параллельно.

Например, если бы вы писали конкурентную хеш-карту, один стресс-тест мог бы иметь N потоков, вставляющих или обновляющих ключи, и M потоков, читающих ключи таким образом, что эти $M+N$ потоков, вероятно, часто выбирают одни и те же ключи. Такой тест не проверяет конкретный исход или значение, но вместо этого пытается запустить как можно больше возможных чередований операций в надежде, что багованные чередования могут себя проявить.

Стресс-тесты во многом похожи на фаззинг-тесты; в то время как фаззинг генерирует много случайных входных данных для данной функции, стресс-тест вместо этого генерирует много случайных расписаний потоков и доступа к памяти. Точно как фаззеры, стресс-тесты поэтому хороши лишь настолько, насколько хороши утверждения (assertions) в вашем коде; они не могут рассказать вам о баге, который не проявляется каким-то легко обнаруживаемым способом, например как сбой утверждения или какой-либо другой вид паники. По этой причине хорошая идея — усеять ваш низкоуровневый конкурентный код утверждениями или `debug_assert_*`, если вы беспокоитесь о стоимости во время выполнения в особенно «горячих» циклах.

Используйте инструменты тестирования конкурентности (Use Concurrency Testing Tools)

Главная сложность в написании конкурентного кода — справиться со всеми возможными способами, которыми может чередоваться выполнение разных потоков. Как мы видели в примере `Ordering::SeqCst` в листинге 11-4, важно не только планирование потоков, но и то, какие значения в памяти возможно наблюдать данному потоку в любой данный момент времени. Написание тестов, которые выполняют каждое возможное допустимое выполнение, не только утомительно, но и сложно — вам нужен очень низкоуровневый контроль над тем, какие потоки выполняются когда и какие значения возвращают их чтения, чего операционная система, скорее всего, не предоставляет.

Проверка моделей с помощью Loom (Model Checking with Loom)

К счастью, уже существует инструмент, который может упростить для вас это исследование выполнений, в форме крейта `loom`. Учитывая относительные циклы выпуска этой книги и крейта `Rust`, я не буду приводить здесь никаких примеров того, как использовать `Loom`, поскольку они, скорее всего, устареют к моменту, когда вы прочтёте эту книгу, но я дам обзор того, что он делает.

`Loom` ожидает, что вы напишете специальные тестовые случаи в форме замыканий и передадите их в модель `Loom`. Модель отслеживает все межпоточные взаимодействия и пытается интеллектуально исследовать все возможные итерации этих взаимодействий путём многократного выполнения замыкания тестового случая. Чтобы обнаруживать и контролировать взаимодействия потоков, `Loom` предоставляет замещающие типы для всех типов в стандартной библиотеке, которые позволяют потокам координироваться друг с другом; это включает большинство типов из `std::sync` и `std::thread`, а также `UnsafeCell` и несколько других. `Loom` ожидает, что ваше приложение использует эти замещающие типы вместо их тёзок всякий раз, когда вы запускаете тесты `Loom`. Замещающие типы привязываются к исполнителю `Loom` и выполняют двойную функцию: они действуют как точки перепланирования, чтобы `Loom` мог выбрать, какую операцию запустить следующей после каждой возможной точки взаимодействия потоков, и они информируют `Loom` о новых чередованиях выполнения, которые стоит рассмотреть. По существу, `Loom` строит дерево всех возможных будущих выполнений для каждой точки, где возможны несколько чередований выполнения, а затем пытается выполнить все из них, одно за другим.

`Loom` пытается полностью исследовать все возможные выполнения тестовых случаев, которые вы ему предоставляете. Хотя это здорово для меньших тестовых случаев, в целом нецелесообразно применять такой вид строгого тестирования к более крупным тестовым случаям, которые тестируют более сложные последовательности операций или требуют запуска многих потоков сразу. `Loom` просто занял бы слишком много времени, чтобы получить приличное покрытие кода. На практике вы поэтому можете захотеть сказать `Loom` рассматривать только подмножество возможных выполнений, о чём в документации `Loom` есть больше деталей.

Как и со стресс-тестами, `Loom` может ловить только баги, которые проявляются как паники, так что это ещё одна причина потратить некоторое

время на размещение стратегических утверждений в вашем конкурентном коде! Во многих случаях может даже стоить добавить в ваш конкурентный код дополнительные инструкции отслеживания состояния и учёта, чтобы дать вам более хорошие утверждения.

Проверка во время выполнения с помощью ThreadSanitizer (Runtime Checking with ThreadSanitizer)

Для более крупных тестовых случаев лучше всего прогнать тест через несколько итераций под превосходным ThreadSanitizer от Google, также известным как TSan. TSan автоматически дополняет ваш код, размещая дополнительные инструкции учёта перед каждым обращением к памяти. Затем, по мере выполнения вашего кода, эти инструкции учёта обновляют и проверяют специальный конечный автомат, который помечает любые конкурентные операции с памятью, указывающие на проблемное состояние гонки. Например, если поток В пишет в некоторое атомарное значение X, но не синхронизировался (здесь много взмахов руками) с потоком, который записал предыдущее значение X, это указывает на гонку запись/запись, что почти всегда баг.

Поскольку TSan только наблюдает за выполнением вашего кода, а не выполняет его снова и снова, как Loom, он лишь добавляет постоянный множитель накладных расходов ко времени выполнения вашей программы. Хотя этот множитель может быть значительным (5–15 на момент написания), он всё же достаточно мал, чтобы вы могли выполнять даже большинство сложных тестовых случаев за разумное время.

На момент написания, чтобы использовать TSan, вам нужно использовать nightly-версию компилятора Rust и передать аргумент командной строки `-Zsanitizer=thread` (или установить его в `RUSTFLAGS`), хотя, надеюсь, со временем это станет стандартной поддерживаемой опцией. Доступны и другие санитайзеры, которые проверяют такие вещи, как обращения к памяти вне границ, использование после освобождения (use-after-free), утечки памяти и чтения неинициализированной памяти, и вы можете захотеть прогнать ваш конкурентный набор тестов и через них!

HEISENBUGS (ГЕЙЗЕНБАГИ) *Гейзенбаги (Heisenbugs)* — это баги, которые, кажется, исчезают, когда вы пытаетесь их изучить. Это случается довольно часто при попытке отлаживать сильно

конкурентный код; дополнительные инструменты для отладки проблемы меняют относительное время конкурентных событий и могут заставить чередование выполнения, которое запускало баг, больше не происходить.

Особенно распространённая причина исчезающих багов конкурентности — операторы печати, использование которых на сегодняшний день является одним из самых распространённых приёмов отладки. Есть две причины, почему операторы печати оказывают такой непропорционально большой эффект на баги конкурентности. Первая, и, пожалуй, самая очевидная, состоит в том, что относительно говоря, требуется довольно долгое время, чтобы напечатать что-то в терминал пользователя (или куда бы ни указывал стандартный вывод), особенно если ваша программа производит много вывода. Запись в терминал требует, как минимум, кругового похода в ядро операционной системы для выполнения записи, но запись может также ждать, пока терминал прочтёт вывод процесса в свои собственные буферы. Всё это дополнительное время может задержать операцию, которая ранее состязалась с операцией в каком-то другом потоке, настолько, что состояние гонки исчезает.

Вторая причина, почему операторы печати нарушают паттерны конкурентного выполнения, в том, что запись в стандартный вывод (в общем случае) охраняется блокировкой. Если вы заглянете внутрь типа `Stdout` в стандартной библиотеке, вы увидите, что он удерживает `Mutex`, который охраняет доступ к выходному потоку. Он делает это для того, чтобы вывод не был слишком сильно искажён, если несколько потоков пытаются писать в одно и то же время — без блокировки данная строка могла бы содержать символы, перемежающиеся из записей нескольких потоков, но с блокировкой потоки будут писать по очереди. К сожалению, захват блокировки вывода — это ещё одна точка синхронизации потоков, и притом такая, в которой участвует каждый печатающий поток. Это означает, что если ваш код ранее был сломан из-за отсутствующей синхронизации между двумя потоками или просто потому, что была возможна определённая гонка между двумя потоками, добавление операторов печати могло бы исправить этот баг как побочный эффект!

продолжение

В общем, когда вы замечаете что-то похожее на гейзенбаг, постарайтесь найти другие способы сузить проблему. Это может включать использование Loom или TSan, или использование внутривещного журнала в памяти, который вы печатаете только в конце. Многие фреймворки логирования также усердно работают над тем, чтобы избежать точек синхронизации на критическом пути выпуска событий журнала, так что переключение на один из них может облегчить вашу жизнь. В качестве дополнительного бонуса хорошее логирование, которое вы оставляете после исправления конкретного бага, может пригодиться позже. Лично я большой поклонник крейта tracing, но есть много хороших вариантов.

Итоги (Summary)

В этой главе мы сначала рассмотрели распространённые подводные камни корректности и производительности в конкурентном Rust, а также некоторые высокоуровневые паттерны конкурентности, которые успешные конкурентные приложения, как правило, используют, чтобы их обойти. Мы также исследовали, как асинхронный Rust обеспечивает конкурентность без параллелизма и как явно вводить параллелизм в асинхронный код Rust. Затем мы глубже погрузились во множество различных низкоуровневых примитивов конкурентности Rust, включая то, как они работают, чем различаются и для чего они все нужны. Наконец, мы исследовали приёмы для написания лучшего конкурентного кода и рассмотрели такие инструменты, как Loom и TSan, которые могут помочь вам проверить этот код. В следующей главе мы продолжим наше путешествие по нижним уровням Rust, углубившись в интерфейсы внешних функций (foreign function interfaces), которые позволяют коду на Rust напрямую связываться с кодом, написанным на других языках.